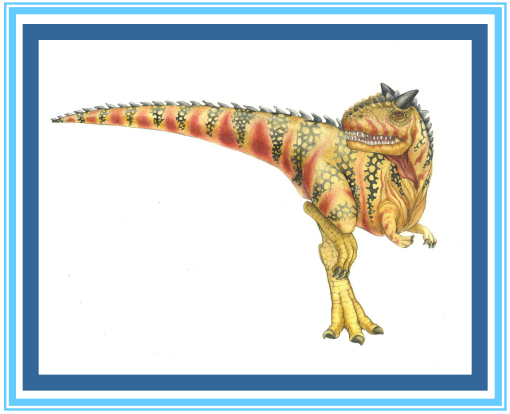
Chapter 7: Deadlocks



Silberschatz, Galvin and Gagne ©2013



Chapter 7: Deadlocks

- 7.1 System Model
- 7.2 Deadlock Characterization
- 7.3 Methods for Handling Deadlocks
- 7.4 Deadlock Prevention
- 7.5 Deadlock Avoidance
- 7.6 Deadlock Detection
- 7.7 Recovery from Deadlock
- 7.8 Summary





Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks.
- To present a number of different methods for preventing or avoiding deadlocks in a computer system.





7.1 System Model

Overview

- System consists of resources
- Resource types $R_1, R_2, \ldots, R_m$
  - *CPU cycles, memory space, I/O devices*
- Each resource type R_i has W_i instances.
  - Eg: *If a system has two CPUs, then the resource type CPU has two instances. Resource type printer may have five instances.*
- Each process utilizes a resource as follows:
  - **request**
  - **use**
  - **release**



Resources

- Resources can be:
  - a **physical** (eg. *Printers, disk drives*) or
  - a **logical** (eg. *A record in a database, semaphores, monitors*)

- Resources are partitioned into several types,

  $R_1, R_2, \ldots, R_m$

  each consisting of some number of identical instances.



…Resources

- A process may utilize a resource in only the following order:
  - **Request**
    - If the request cannot be granted immediately, then the requesting process must **wait** until it can acquire the resource.
  - **Use**
    - The process can operate on the resource
  - **Release**
    - The process releases the resource



Deadlock

- Many programs competing for limited resources
- **Deadlock** : a state when every process in the set of processes is waiting for an event that can be caused only by another process in the set.



- Example 1:
  - System has 2 CD drives.
  - P_1 and P_2 each hold one CD drive and each needs another one.





System Model

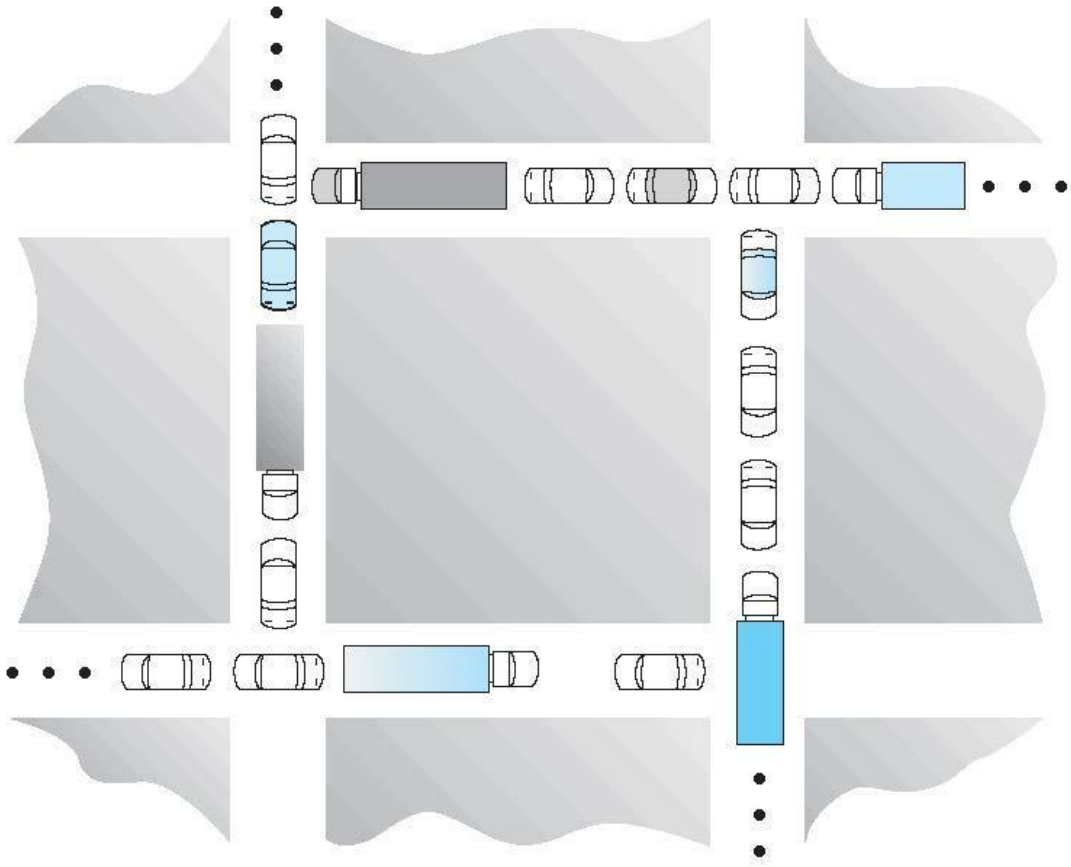






Figure Deadlock example



7.2 Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion**: only one process at a time can use a resource
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Deadlock Characterization

Resource-Allocation Graph (RAG)

A set of vertices V and a set of edges E.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$



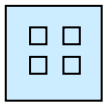


7.2 Deadlock Characterization

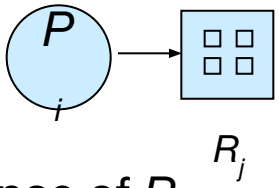
- Process



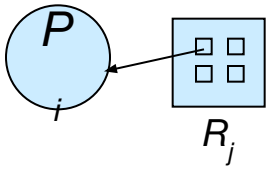
- Resource Type with 4 instances



- P_i requests instance of R_j



- P_i is holding an instance of R_j



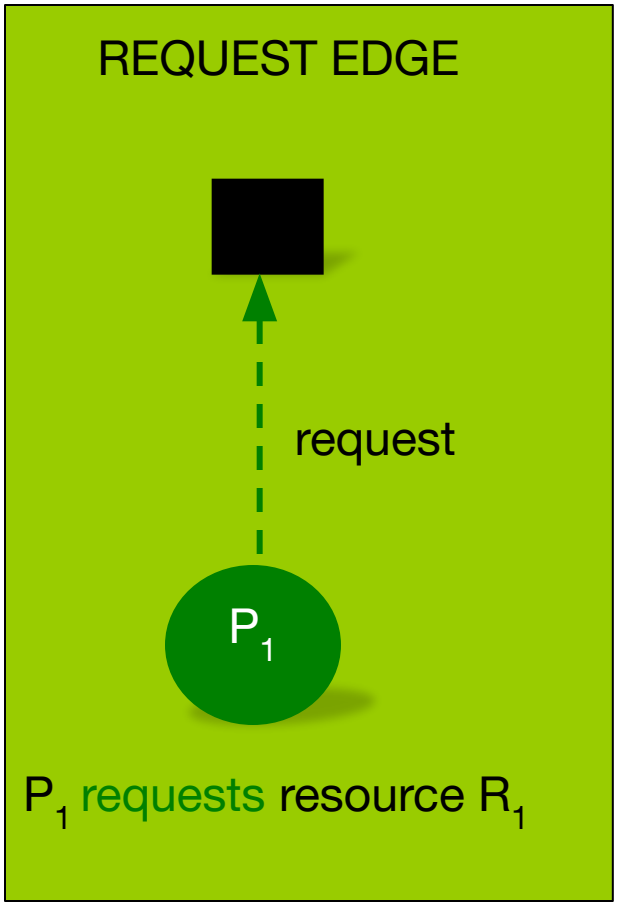


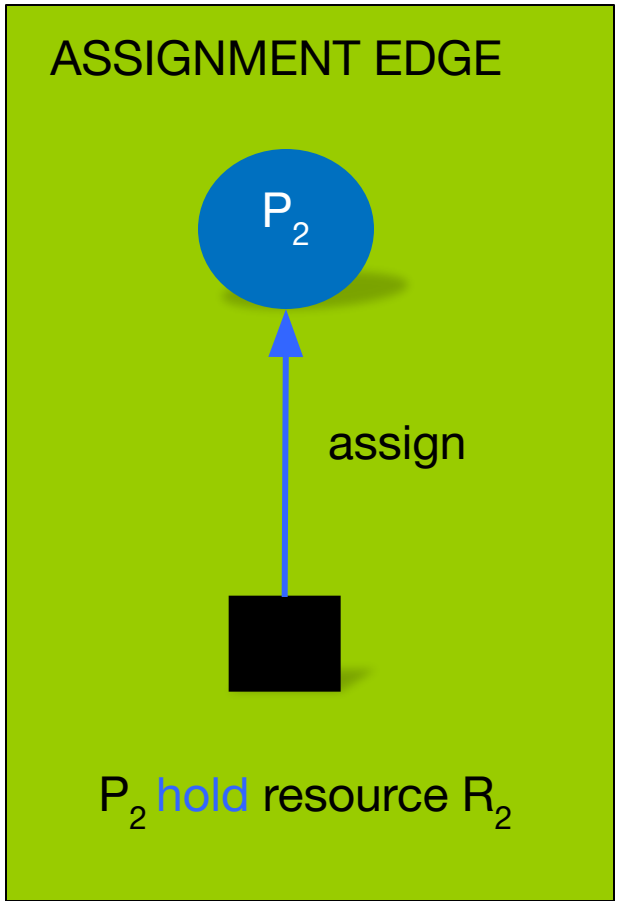


7.1

Deadlock Characterization

- Type of edges, E





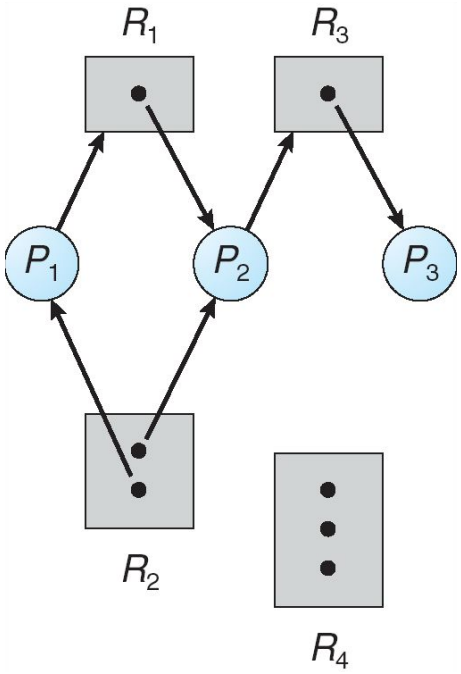
Request edge:
directed edge $P_i \rightarrow R_j$

Assignment edge:
directed edge $R_j \rightarrow P_i$





7.2 Deadlock Characterization



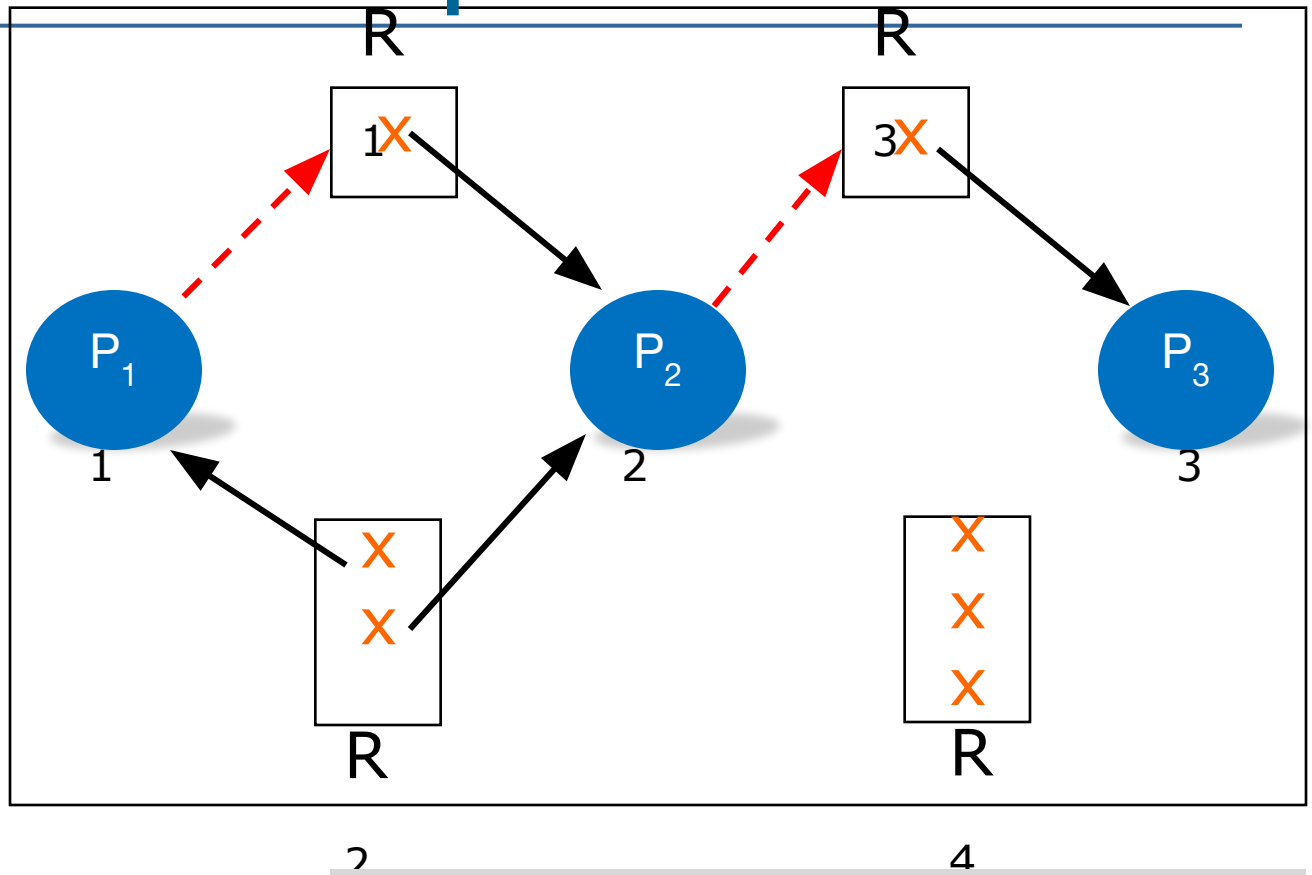
Example of a Resource Allocation Graph



Example 1

RAG:

(Several Instances of Resources)



Set **P**, **R** and **E**:

- **P** = {P_1 , P_2 , P_3}
- **R** = {R_1 , R_2 , R_3}
- **E** = {P_1 □ R_1 , P_2 □ R_3 ,
  R_1 □ P_2 , R_2 □ P_2 ,
  R_2 □ P_1 , R_3 □ P_3}

Process state:

- P_1 holds R_2 and waits for R_1
- P_2 holds R_1 and R_2, and waits for R_3,
- P_3 holds R_3

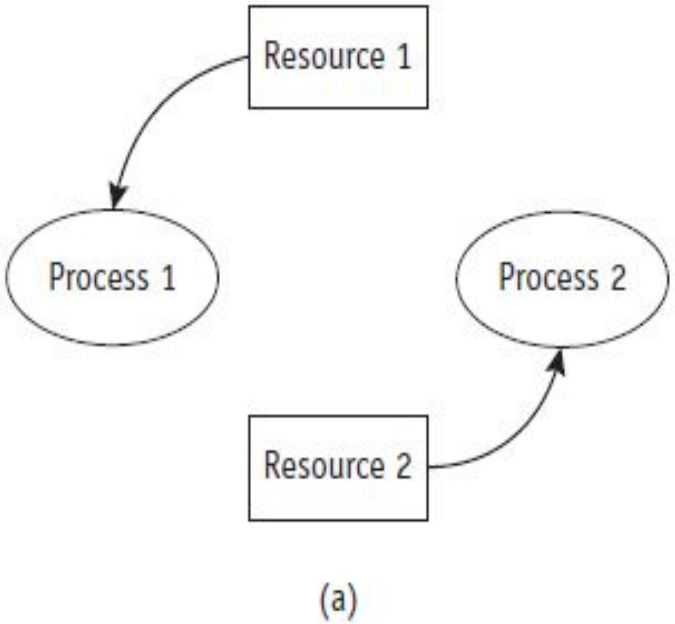


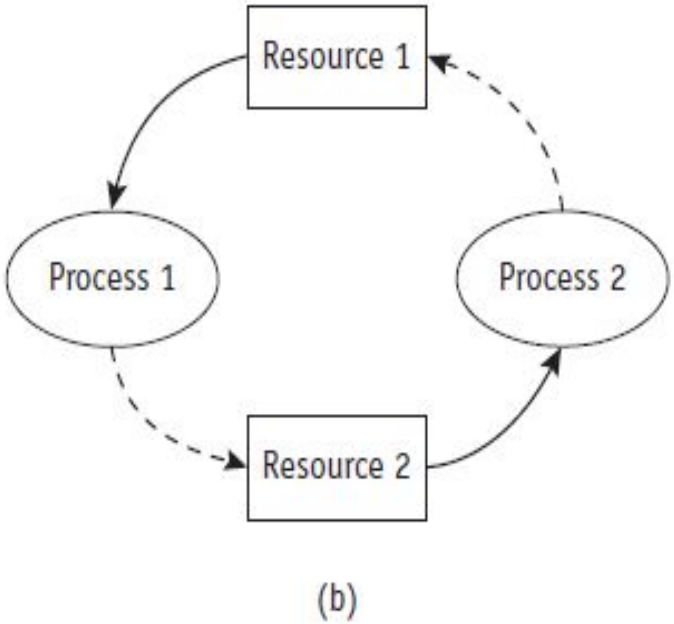
Basic Facts

(a) If graph contains no cycles ⇒ no deadlock

(b) If graph contains a cycle ⇒

- If only one instance per resource type, then deadlock exist
- If several instances per resource type, then possibility of deadlock

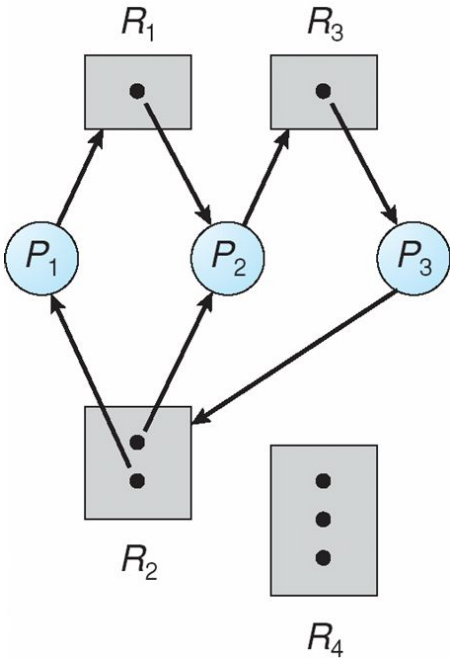








7.2 Deadlock Characterization



Is there a deadlock?

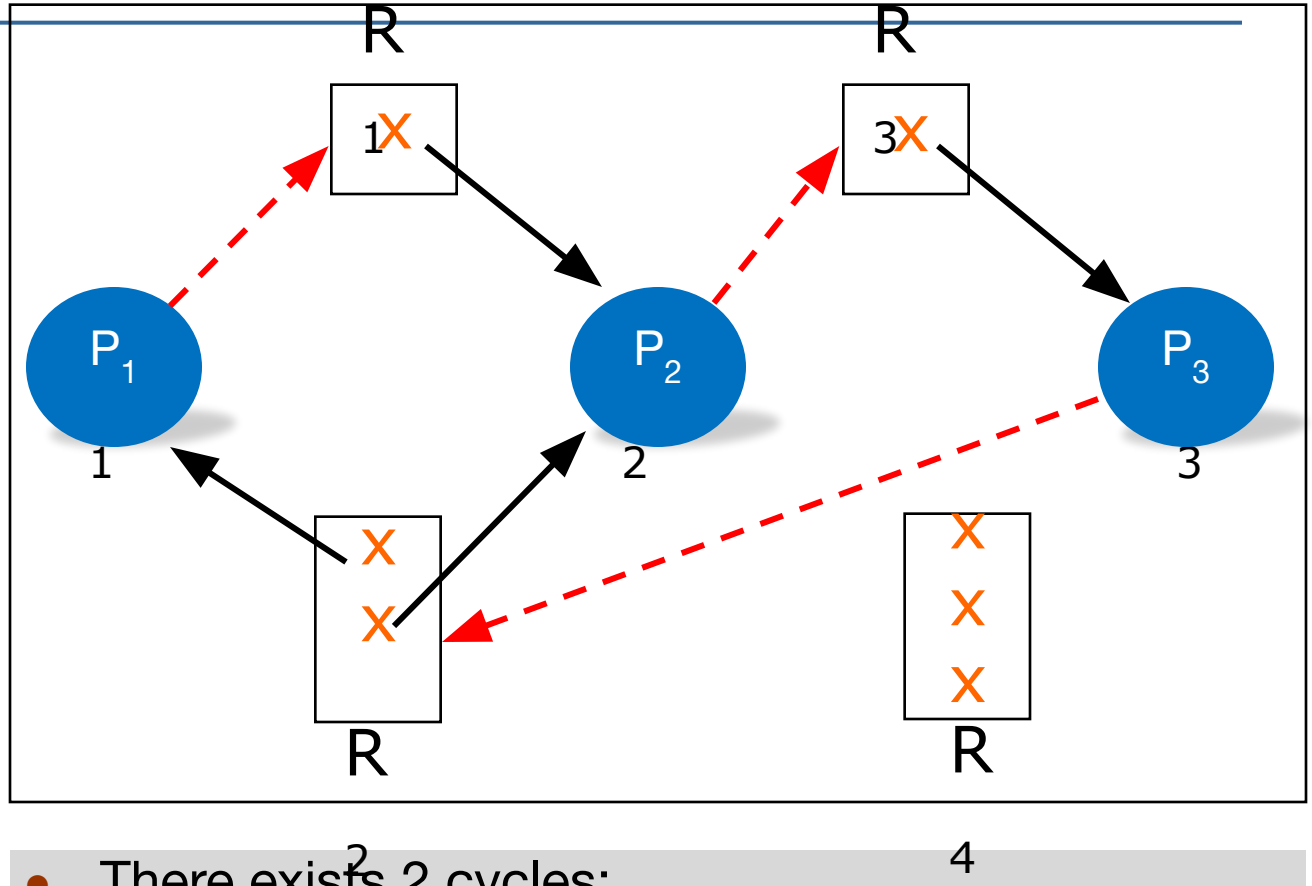
Resource Allocation Graph With a Cycle





RAG: Example 2

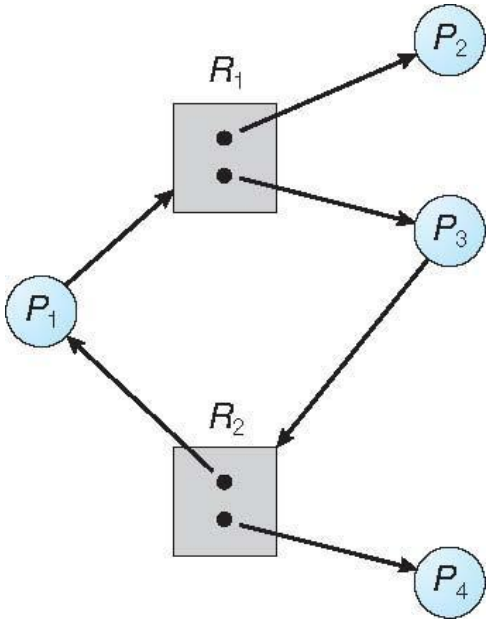
(Several Instances of Resources)



- There exists 2 cycles:
  - $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
  - $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$
- P_1 , P_2 and P_3 are in a deadlock state



7.2 Deadlock Characterization



Is there a deadlock?

Resource Allocation Graph With a Cycle

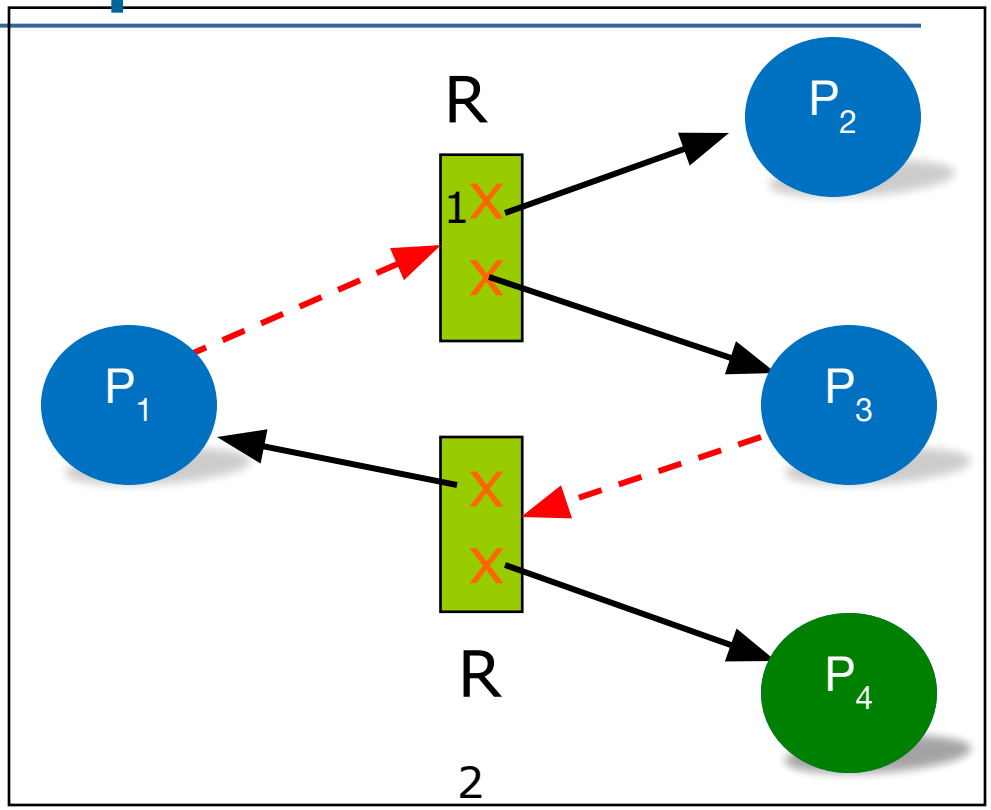




Example 3

RAG:

- However, the system is not deadlock:
  - when P_4 releases R_2, this resource can be assigned to P_3



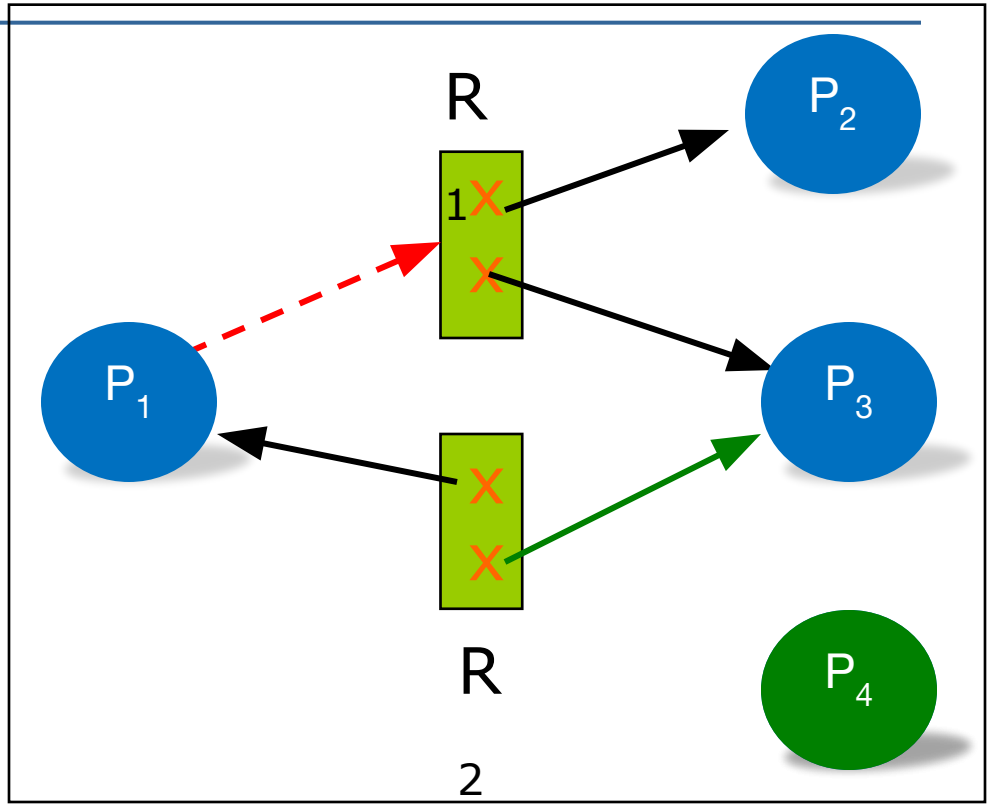
- The RAG has one cycle: $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- P_1 and P_3 are in a deadlock state





RAG: Example 3

- However, the system is not deadlock:
  - when P_4 releases R_2, this resource can be assigned to P_3

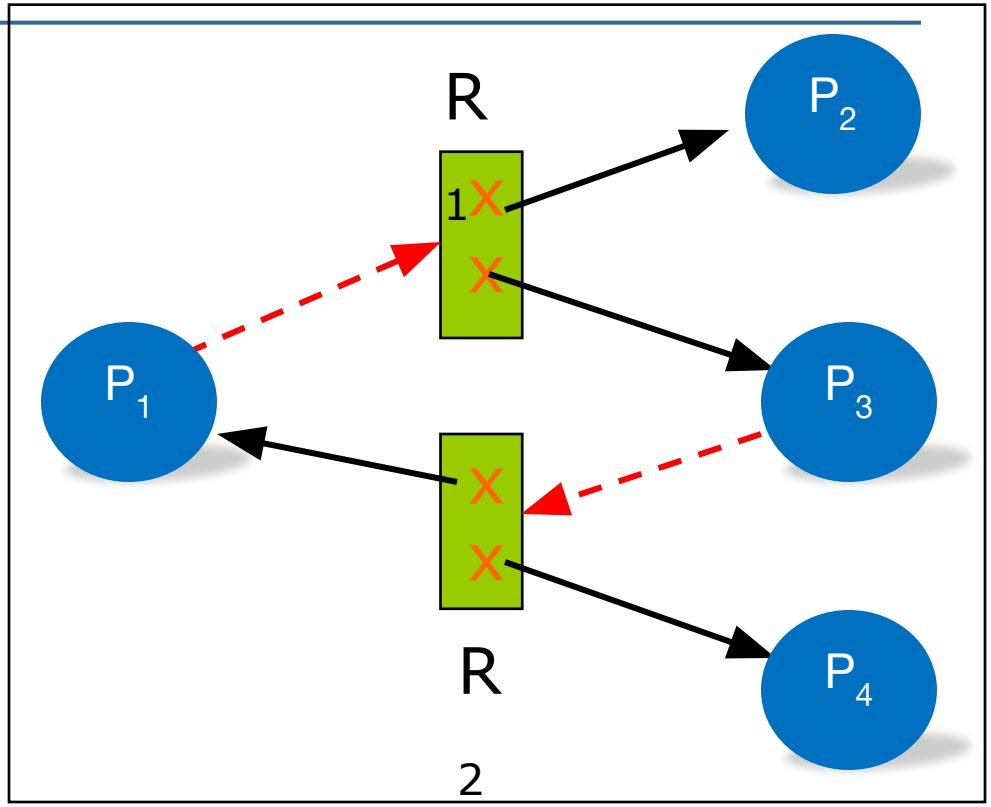


No more cycle:



Exercise

- Write the set of P, R and E.



- The RAG has one cycle:
  $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- P_1 and P_3 are in a deadlock state



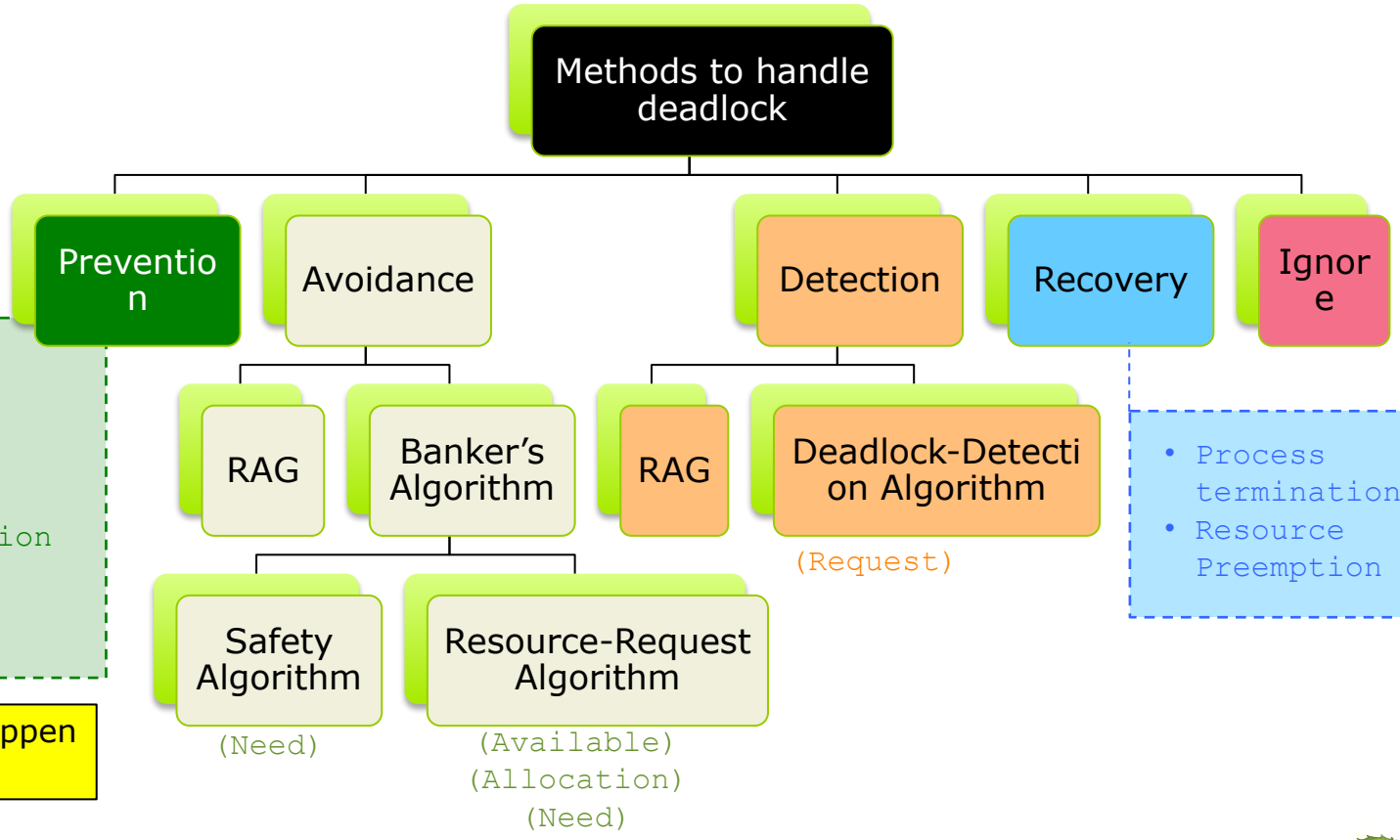
7.3 Methods for Handling Deadlocks

- 1. Ensure that the system will ***never*** enter a deadlock state:
  - Deadlock prevention
  - Deadlock avoidance
- 2. Allow the system to enter a deadlock state and then recover
- 3. Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX





7.3 Methods for Handling Deadlocks







7.4 Deadlock Prevention

- Ensure that at least one of the necessary condition for deadlocks does not hold. Can be accomplished restraining the ways request can be made
  - **Mutual Exclusion** – Must hold for non-sharable resources that can be accessed simultaneously by various processes. Therefore cannot be used for prevention.
  - **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
    - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
    - Low resource utilization; starvation possible





7.4 Deadlock Prevention

- **No Preemption** – not practical for most systems
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
  - Preempted resources are added to the list of resources for which the process is waiting
  - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration. Can be used in practice.





Deadlock Prevention

- Two bank transactions 1 and 2 execute concurrently.
  - Transaction 1 transfers $25 from account A to account B
  - Transaction 2 transfers $50 from account B to account A
- Program

```
void transaction(Account from,
Account to, double amount)
{
   mutex lock1, lock2;
   lock1 = get_lock(from);
   lock2 = get_lock(to);
   acquire(lock1);
      acquire(lock2);
         withdraw(from, amount);
         deposit(to, amount);
      release(lock2);
   release(lock1);
}
```

Deadlock Example with Lock Ordering





7.5 Deadlock Avoidance

- Ensure that the system will ***never*** enter a deadlock state
  - What is a trivial way of avoiding deadlocks?
- Requires that the system to have some additional ***a priori*** information available on possible resource requests.
  - Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
  - The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
  - Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes





7.5 Deadlock Avoidance

Safe state

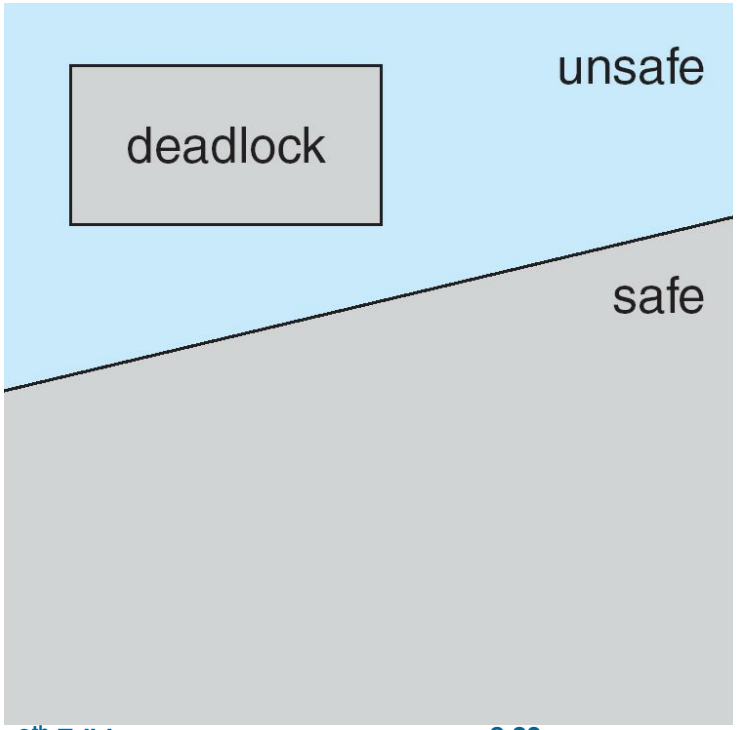
- System is in **safe state** if there exists a sequence $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the systems such that for each P_i the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < i$
- That is:
  - If P_i resource needs are not immediately available, then P_i can wait until all processes P_j ($j < i$) have finished executing.
  - When they have finished executing they release all their resources and then P_i can obtain the needed resources, execute, return allocated resources, and terminate
  - When P_i terminates, P_{i+1} can obtain its needed resources, and so on





7.5 Deadlock Avoidance – Safe State

- System is in **safe state** if the system can allocate resources to each process (up to maximum) in some order and still avoid the deadlock.
- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ possibility of deadlock



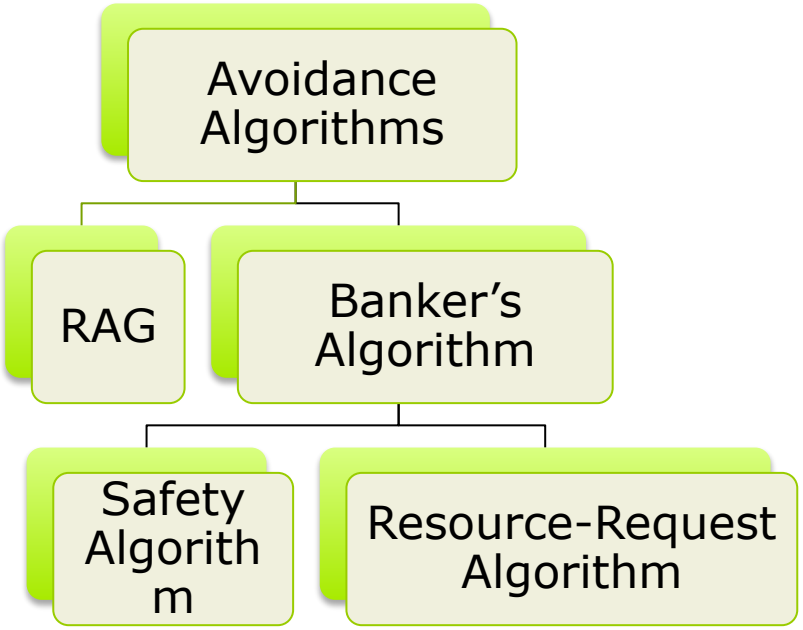
Basic Facts



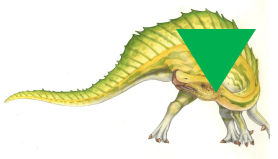


Deadlock Avoidance

Algorithms



- All processes can be started any time.
- When a process requests for a resource, an **avoidance algorithm** is executed to check whether a deadlock may occur in future.
- If deadlock may occur, the **request is delayed**.





7.5 Deadlock Avoidance Algorithms

- Deadlock avoidance ⇒ ensure that a system will never enter an unsafe state.

1. Single instance of a resource type
   - Use a variant of the resource-allocation graph
2. Multiple instances of a resource type
   - Use the banker's algorithm





Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- Single instance of a resource type
- Each process must a priori claim maximum resource use
- Use a variant of the resource-allocation graph with claim edges.
- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system
- A cycle in the graph implies that the system is in unsafe state



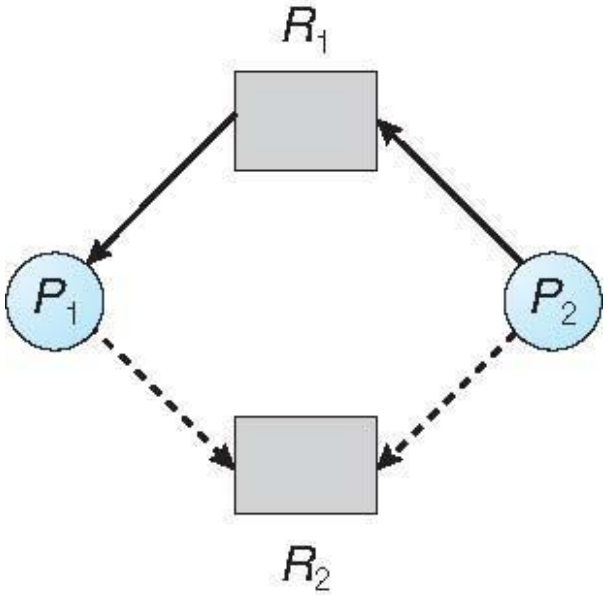


Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- P1 is holding resource R1 and has a claim on R2
- P2 is requesting R1 and has a claim on R2
- No cycle. So system is in a safe state.

Resource-Allocation Graph with claim edges



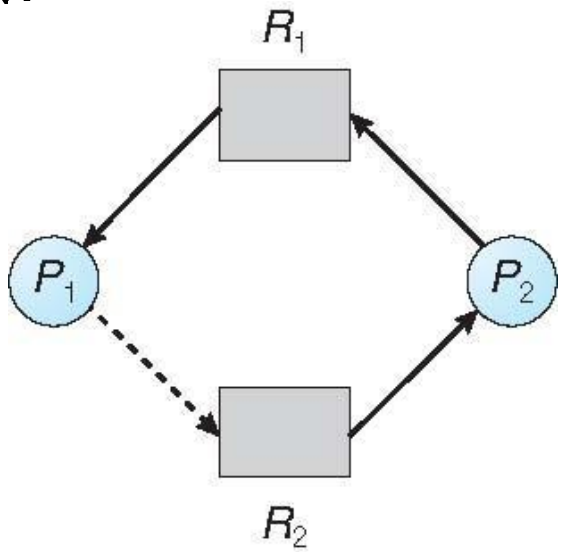




Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- The graph of slide before with a claim edge from P2 to R2 is changing to an assignment edge.
- There is a cycle in the graph □ unsafe state.
- Is there a deadlock?



Example of a Resource Allocation Graph





Deadlock Avoidance

(1) Resource Allocation Graph (RAG) scheme

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph
- Otherwise, the process must wait





7.5

Deadlock Avoidance

(2) Banker's algorithm

- Multiple instances of a resource type
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time.

"The algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers"





Deadlock Avoidance

(2) Banker's algorithm

Example 1: Bank Loan

- Bank with fixed amount of capital which operate as follows:
  - No customer will be granted a loan exceeding the bank's total capital
  - All customers will be given **maximum credit limit** when opening an account
  - The sum of all loan must not exceed the bank's total capital





7.5

Deadlock Avoidance

(2) Banker's algorithm

. . . Example 1a

- The bank total capital fund is RM100,000 and has 3 customers C_1, C_2, and C_3 each with credit limits RM40,000, RM 50,000, and RM80,000.

Customer	Loan Amount	Max Credit	Remaining Credit
C_1	0	40,000	40,000
C_2	20,000	50,000	30,000
C_3	40,000	80,000	40,000
Total loan	60,000		
Total capital fund	100,000	Available fund	40,000

- safe state the bank has enough money left to satisfy customer's need (remaining credit)





Deadlock Avoidance

(2) Banker's algorithm

. . . Example 1b

- C_1 and C_2 made a loan of RM20,000 and RM10,000

Customer	Loan Amount	Max Credit	Remaining Credit
C_1	20,000	40,000	20,000
C_2	30,000	50,000	20,000
C_3	40,000	80,000	40,000
Total loan	90,000		
Total capital fund	100,000	Available fund	10,000

- □ unsafe state, the bank doesn't have enough money left to satisfy customer's need (available fund < min remaining credit)





Deadlock Avoidance

(2) Banker's algorithm

Data structures of Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m.
  - If available [j] = k, then there are k instances of resource type R_j available
- **Max**: n x m matrix.
  - If Max [i,j] = k, then process P_i may request at most k instances of resource type R_j
- **Allocation**: n x m matrix.
  - If Allocation[i,j] = k then P_i is currently allocated k instances of R_j
- **Need**: n x m matrix.
  - If $Need$[i,j] = k, then P_i may need at most k more instances of R_j to complete its task.
    - $Need$ [i,j] = Max[i,j] – $Allocation$ [i,j]





Deadlock Avoidance

2a Safety algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work = Available

   Finish [i] = false **for** ***i*** = 0, 1, …, n- 1

2. Find an ***i*** such that both:

   (a) ***Finish [i] = false***

   (b) $\boldsymbol{Need_i \leq Work}$

   If no such ***i*** exists, go to step 4

3. $\boldsymbol{Work = Work + Allocation_i}$

   Finish[i] = true

   go to step 2

4. If ***Finish [i] == true*** for all ***i***, then the system is in a safe state. Otherwise, in an unsafe state.





7.5 Example of Safety Algorithm - 1

- 5 processes -- P_0 ….. P_4;
- 3 resource types:

  A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time T_0:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

Available = Total Resources – Total Allocation Resources
= [A B C] – [A B C]
= [10 5 7] – [7 2 5]
= [3 3 2]



Example of Safety Algorithm - 2

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	3	3	2
P1	2	0	0	1	2	2			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			





Example of Safety Algorithm - 3

Step 1: **Work** = **Available** = (3, 3, 2)

Step 2: { **Need**(P_0) ≤ **Work** }

Step 2: { **Need**(P_1) ≤ **Work** }

{ (1, 2, 2) ≤ (3, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_1)

= (3, 3, 2) + (2, 0, 0)

= (5, 3, 2)

Step 2: { **Need**(P_2) ≤ **Work** }

Step 3: { **Need**(P_3) ≤ **Work** }

{ (0, 1, 1) ≤ (5, 3, 2) }

Work = **Work** + **Allocation**(P_3)

= (5, 3, 2) + (2, 1, 1)

= (7, 4, 3)

(……Repeat for all other processes)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1





Example of Safety Algorithm - 4

Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 4, 5) + (3, 0, 2)
= (10, 4, 7)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (10, 4, 7) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (10, 4, 7) + (0, 1, 0)
= (10, 5, 7)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

What is the safe sequences produced?

- The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria





Deadlock Avoidance

2b Resource-Request algorithm

Let $Request_i$ [...] be the request vector for process P_i.

$Request_i[j] = k$. Process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

- If safe ⇒ the resources are allocated to P_i
- If unsafe ⇒ P_i must wait, and the old resource-allocation state is restored





Example Resource-Request Algorithm - 1

- 5 processes P_0 through P_4;

  3 resource types:

  A (10 instances), *B* (5 instances), and *C* (7 instances)

- Snapshot at time T_0:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			





Example Resource-Request Algorithm - 2 (Determine Safe State)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

Step 1: **Work** = **Available** = (3, 3, 2)

Step 2: { **Need**(P_0) ≤ **Work** }

Step 2: { **Need**(P_1) ≤ **Work** }

{ (1, 2, 2) ≤ (3, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_1)

= (3, 3, 2) + (2, 0, 0)

= (5, 3, 2)

Step 2: { **Need**(P_2) ≤ **Work** }

Step 3: { **Need**(P_3) ≤ **Work** }

{ (0, 1, 1) ≤ (5, 3, 2) }

Work = **Work** + **Allocation**(P_3)

= (5, 3, 2) + (2, 1, 1)

= (7, 4, 3)

(……Repeat for all other processes)



Example Resource-Request Algorithm – 3 (Determine Safe State)



Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 4, 5) + (3, 0, 2)
= (10, 4, 7)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (10, 4, 7) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (10, 4, 7) + (0, 1, 0)
= (10, 5, 7)

	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3
P_1	2	0	0	3	2	2	1	2	2
P_2	3	0	2	9	0	2	6	0	0
P_3	2	1	1	2	2	2	0	1	1
P_4	0	0	2	4	3	3	4	3	1

What is the safe sequences produced?

- **The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria – Get using Safety Algorithm**





Example Resource-Request Algorithm - 4

Example: P_1 Request (1,0,2)

- Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true
- Since **Request$_1$ ≤ Need$_i$** & **Request$_1$ ≤ Available**, pretend this request has been fulfilled □ execute *Resource-Request Algorithm*.

Available

= **Available** – **Request**(P_1)

= (3, 3, 2) – (1, 0, 2)

= **(2, 3, 0)**

Allocation(P_1)

= **Allocation**(P_1) + **Request**(P_1)

= (2, 0, 0) + (1, 0, 2)

= **(3, 0, 2)**

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

Need(P_1) = **Need**(P_1) – **Request**(P_1)

= (1, 2, 2) – (1, 0, 2)

= **(0, 2, 0)**





Example Resource-Request Algorithm - 5

Example: P_1 Request (1,0,2)

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

Step 1: **Work** = **Available** = (2, 3, 0)

Step 2: { **Need**(P_1) ≤ **Work** }
{ (0, 2, 0) ≤ (2, 3, 0) }

Step 3: **Work** = **Work** + **Allocation**(P_1)
= (2, 3, 0) + (3, 0, 2)
= (5, 3, 2)

Step 2: { **Need**(P_3) ≤ **Work** }
{ (0, 1, 1) ≤ (5, 3, 2) }

Step 3: **Work** = **Work** + **Allocation**(P_3)
= (5, 3, 2) + (2, 1, 1)
= (7, 4, 3)





Example Resource-Request Algorithm - 6
Example: P_1 Request (1,0,2)

Step 2: { **Need**(P_4) ≤ **Work** }
{ (4, 3, 1) ≤ (7, 4, 3) }
Step 3: **Work** = **Work** + **Allocation**(P_4)
= (7, 4, 3) + (0, 0, 2)
= (7, 4, 5)

Step 2: { **Need**(P_0) ≤ **Work** }
{ (7, 4, 3) ≤ (7, 4, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_0)
= (7, 4, 5) + (0, 1, 0)
= (7, 5, 5)

Step 2: { **Need**(P_2) ≤ **Work** }
{ (6, 0, 0) ≤ (7, 5, 5) }
Step 3: **Work** = **Work** + **Allocation**(P_2)
= (7, 5, 5) + (3, 0, 2)
= (10, 5, 7)

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

What is the new safe sequences ?

Executing algorithm shows that sequence < P_1, P_3, P_4, P_0, P_2> satisfies safety requirement

The system can grant the request of process P_1.





Example Resource-Request Algorithm - 7

Example: P_4 Request (3,3,0)

- Check that Request ≤ Available (that is, (3,3,0) ≤ (2,3,0) ⇒ false

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	7	4	3	**2**	**3**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

- **P_4 cannot be granted, since the resources are not available. $Request_4 \leq Available$ becomes FALSE, this request has been not fulfilled.**





Example Resource-Request Algorithm - 8

Example: P_0 Request (0,2,0)

- Check that Request ≤ Available (that is, (0,2,0) ≤ (2,3,0) ⇒ true
- Since **Request$_0$ ≤ Need$_i$** & **Request$_1$ ≤ Available**, pretend this request has been fulfilled □ execute *Resource-Request Algorithm*.

Available

= **Available** – **Request**(P_0)

= (2, 3, 0) – (0, 2, 0)

= **(2, 1, 0)**

Allocation(P_0)

= **Allocation**(P_0) + **Request**(P_0)

= (0, 1, 0) + (0, 2, 0)

= **(0, 3, 0)**

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	**0**	**3**	**0**	7	5	3	**7**	**2**	**3**	**2**	**1**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

Need(P_0) = **Need**(P_0) – **Request**(P_0)

= (7, 4, 3) – (0, 2, 0)

= **(7, 2, 3)**





Example Resource-Request Algorithm - 9

Example: P_0 Request (0,2,0)

Step 1: **Work** = **Available** = (2, 1, 0)

	Allocation			Max			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	**0**	**3**	**0**	7	5	3	**7**	**2**	**3**	**2**	**1**	**0**
P_1	**3**	**0**	**2**	3	2	2	**0**	**2**	**0**			
P_2	3	0	2	9	0	2	6	0	0			
P_3	2	1	1	2	2	2	0	1	1			
P_4	0	0	2	4	3	3	4	3	1			

What is the new safe sequences produced?

The request results unsafe state, although resources are available.

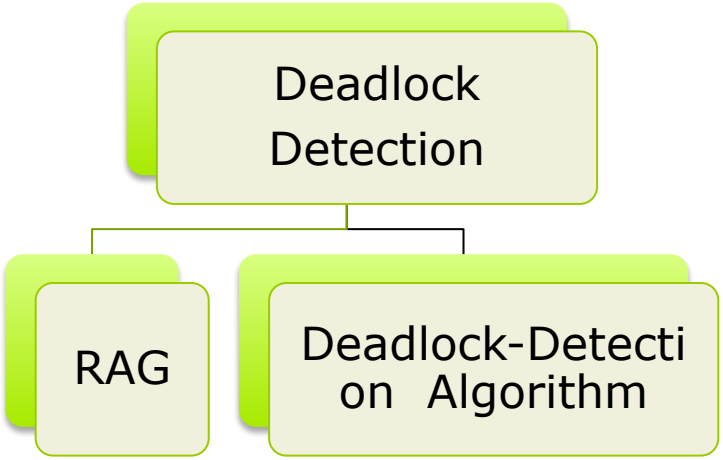
System will not grant the request of process P_0





Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
  - Single instance of a resource type
  - Multiple instances of a resource type.
- Recovery scheme



(almost similar to Safety Algorithm)

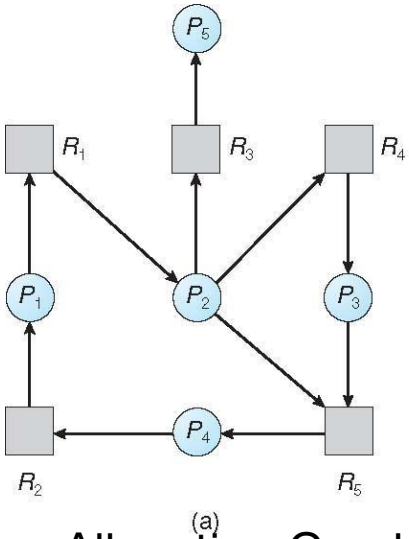




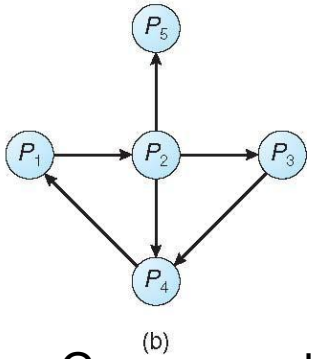
Deadlock Detection

Single Instance of Each Resource Type

- Deadlock can be detected by building *Resource-Allocation Graphs* (RAG)
- Maintain a **wait-for** graph
  - Nodes are processes
  - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Converting a resource-allocation graph to a wait-for graph.



Resource-Allocation Graph



Corresponding wait-for graph





Deadlock Detection

Single Instance of Each Resource Type

- Maintain a **wait-for** graph
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph





Deadlock Detection

Example

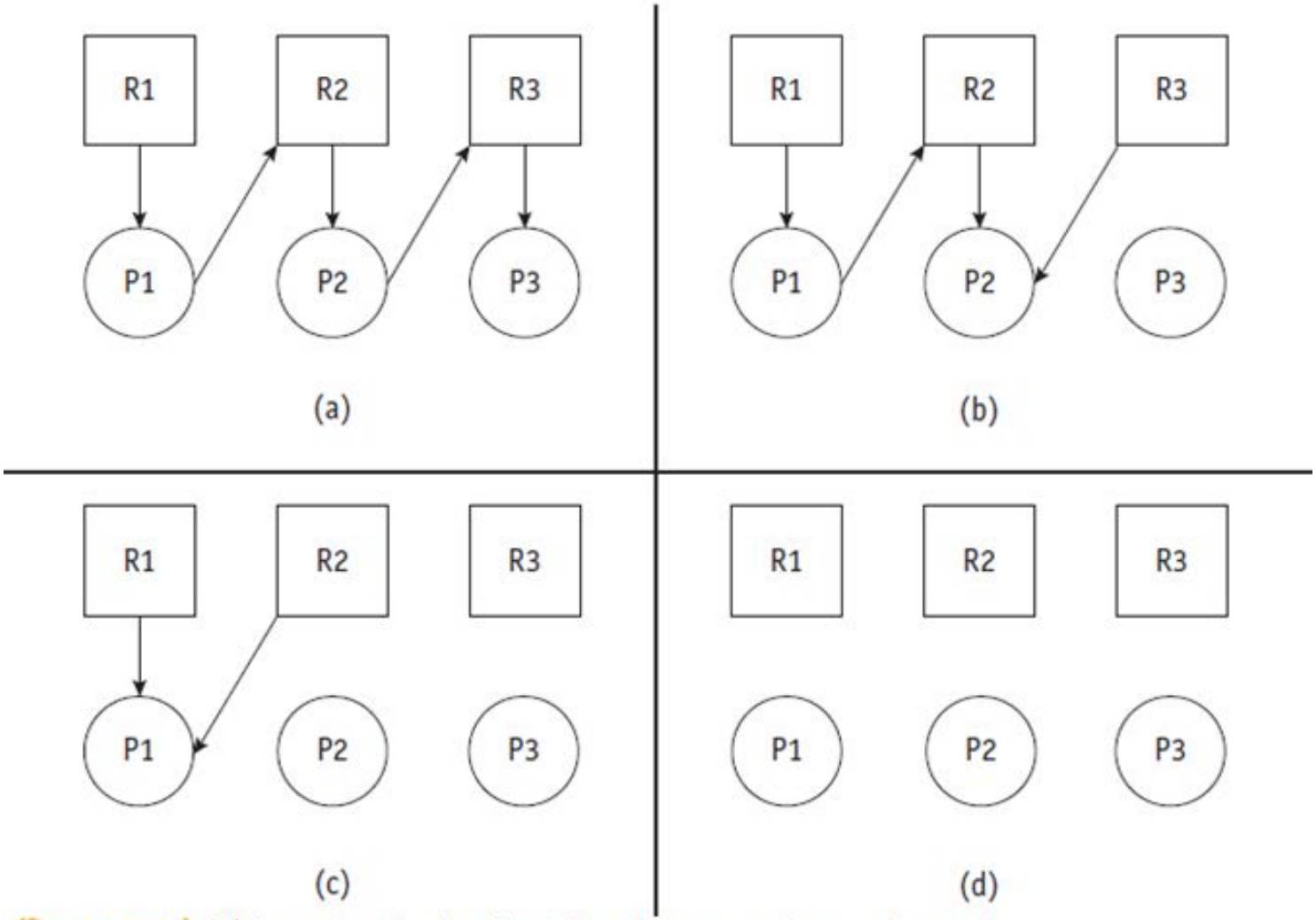
- The detection algorithm can be explained by using RAG and “reducing” them.
- The steps to reduce a graph:

1) Find process using a resource & not waiting for one (*Figure 5.12(b)*).
2) Find a process that’s waiting only for resource that aren’t fully allocated (*Figure 5.12(c)*).
3) Repeat step (1) and (2) until all lines removed (*Figure 5.12(d)*)





7.6 Deadlock Detection



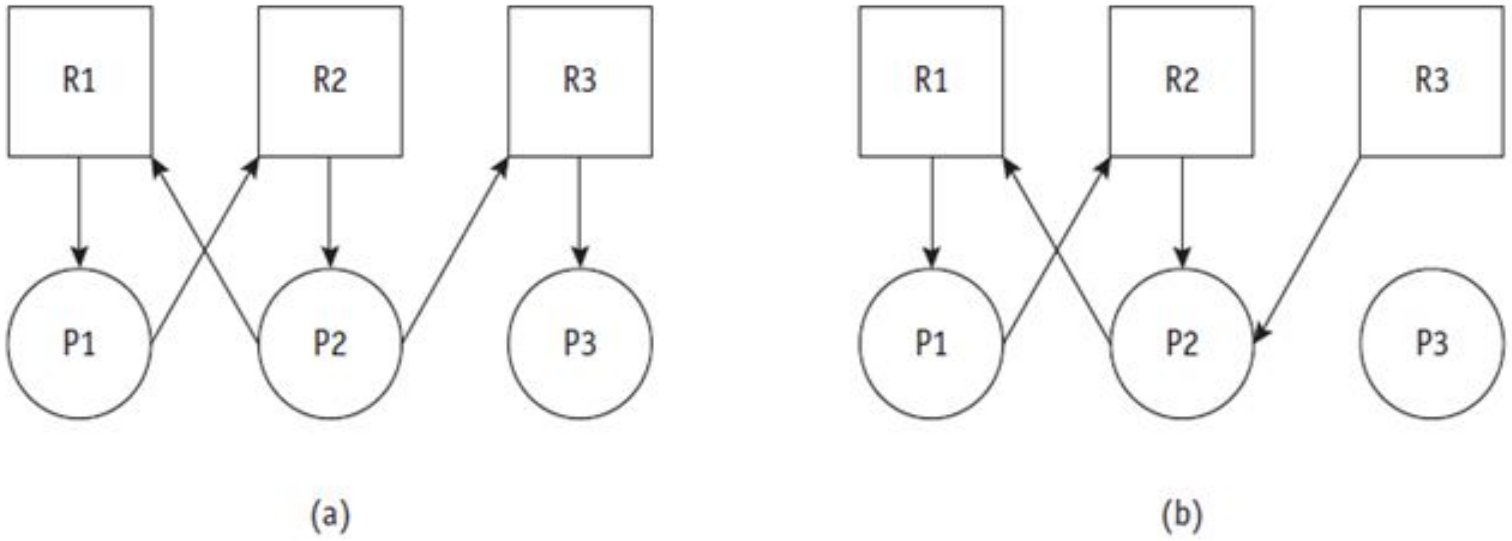
(figure 5.12) *This system is deadlock-free because the graph can be completely reduced, as shown in (d).*



Source: A.M. McHoes and I.M. Flyn, Understanding Operating Systems: 6th Edition, Course Technology, Cengage Learning, United State of America (2011)

Deadlock Detection

- Example a graph remaining the deadlock using those steps:



(figure 5.13) *Even after this graph (a) is reduced as much as possible (by removing the request from P3), it is still deadlocked (b).*



Deadlock Detection

Several/Multiple Instances of a Resource Type

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m. If available [j] = k, then there are k instances of resource type R_j available
- **Allocation**: n x m matrix. If Allocation[i,j] = k, then P_i is currently allocated k instances of R_j
- **Request**: n x m matrix that indicates the current request of each process. If Request[i,j] = k, then process P_i is requesting k additional instances of resource type R_j.





Deadlock Detection

Detection Algorithm

Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:

1. Initialization
   (a) ***Work = Available***
   (b) For ***i*** = 1,2, …, n, if $Allocation_i \neq 0$, then ***Finish*[i] = *false***; otherwise, ***Finish*[i] = *true***
2. Find an index ***i*** such that both:
   (a) ***Finish*[*i*] == *false***
   (b) $Request_i \leq Work$
   If no such ***i*** exists, go to step 4
3. $Work = Work + Allocation_i$
   Finish*[*i*] = *true
   go to step 2
4. If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then P_i is deadlocked





7.6 Example of Detection Algorithm

- Five processes P_0 through P_4; three resource types A (7 instances), *B* (2 instances), and *C* (6 instances)
- Snapshot at time T_0:

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	2	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

- Sequence <P_0, P_2, P_3, P_1, P_4> will result in ***Finish[i] = true*** for all ***i***



	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	2	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

Available		
A	B	C
0	0	0

Step 1:
Work = Available
Work = (0,0,0)
Check all P_i Allocation.
Is it Allocation (P_i) ≠ 0? □ false

Step 2: Request start with P_0
Is Request(P_0) <= Work?
(0,0,0) P_0 <= (0,0,0) □**Yes**
Go to Step 3

Step 3: Update the latest Work
Work = Work + Allocation(P_0)
= (0,0,0) + (0,1,0)
= (0,1,0)
Finish(P_0) = true (Request is fulfilled)
Go to Step 2

Step 2: Request start with P_1
Is Request(P_1) <= Work?
(2,0,2)P_1 <= (0,1,0) □**No, check next P_i**

Step 2: Request start with P_2
Is Request(P_2) <= Work?
(0,0,0)P_2 <= (0,1,0) □**Yes**
Go to Step 3

Step 3: Update the latest Work
Work = Work + Allocation(P_2)
= (0,1,0) + (3,0,2)
= (3,1,2)
Finish(P_2) = true (Request is fulfilled)
Go to Step 2



	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	2	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

Available		
A	B	C
0	0	0

Step 2: Request start with P_3
Is Request(P_3) <= Work?
(1,0,0)P_3 <= (3,1,2) **Yes**
Go to Step 3

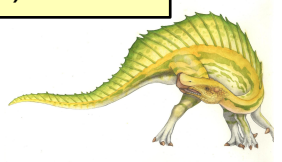
Step 3: Update the latest Work
Work = Work + Allocation(P_3)
= (3,1,2) + (2,1,1)
= (5,2,3)
Finish(P_3) = true (Request is fulfilled)
Go to Step 2

Step 2: Request start with P_4
Is Request(P_4) <= Work?
(0,0,2)P_4 <= (5,2,3) **Yes**
Go to Step 3

Step 3: Update the latest Work
Work = Work + Allocation(P_4)
= (5,2,3) + (0,0,2)
= (5,2,5)
Finish(P_4) = true (Request is fulfilled)
Go to Step 2

Step 2: Request start with P_1
Is Request(P_1) <= Work?
(2,0,2)P_1 <= (5,2,5) **Yes**
Go to Step 3

Step 3: Update the latest Work
Work = Work + Allocation(P_1)
= (5,2,5) + (2,0,0)
= (7,2,5)
Finish(P_1) = true (Request is fulfilled)





7.6 Example of Detection Algorithm (Cont.)

- P_2 requests one additional instance of type **C**

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	2	0	0	1			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

- State of system? – System in deadlock state
  - Can reclaim resources held by process P_0, but insufficient resources to fulfill other processes requests
  - Deadlock exists, consisting of processes P_1, P_2, P_3, and P_4



	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	2	0	0	1			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

P_2 requests one additional instance of type ***C***

Step 1:
Work = Available
Work = (0,0,0)
Check all P_i Allocation.
Is it Allocation (P_i) ≠ 0? □ false

Step 2: Request start with P_0
Is Request(P_0) <= Work?
(0,0,0) P_0 <= (0,0,0) □**Yes**
Go to Step 3

Step 3: Update the latest Work
Work = Work + Allocation(P_0)
= (0,0,0) + (0,1,0)
= (0,1,0)
Finish(P_0) = true (Request is fulfilled)
Go to Step 2

Step 2: Request start with P_1
Is Request(P_1) <= Work?
(2,0,2) P_1 <= (0,0,0) □**No, check P_2**

Step 2: Request start with P_2
Is Request(P_2) <= Work?
(0,0,1) P_2 <= (0,1,0) □**No, check P_3**

Step 2: Request start with P_3
Is Request(P_3) <= Work?
(1,0,0) P_3 <= (0,1,0) □**No, check P_4**

Step 2: Request start with P_4
Is Request(P_4) <= Work?
(0,0,2) P_4 <= (0,1,0) □**No**
If no such P_i exists, go to step 4

Step 4:
If ***Finish(P_i)== false***, for some ***i***, $1 \le i \le 4$, then the system is in deadlock state.





Deadlock Detection

Detection-Algorithm Usage

- If a deadlock is detected we must abort (rollback) some of the processes involved in the deadlock (see next slide)
- Need to decide when, and how often, to invoke the deadlock detection algorithm, which depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.





7.7 Recovery from Deadlock

Process termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
  - Priority of the process
  - How long process has computed, and how much longer to completion
  - Resources the process has used
  - Resources process needs to complete
  - How many processes will need to be terminated
  - Is process interactive or batch?





7.7

Recovery from Deadlock

Resource preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor





7.8 Summary

- A deadlock state occurs when two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes.

- Three principal methods for dealing with deadlocks:
  1. Use some protocol to prevent or avoid deadlocks.
  2. Allow the system to enter a deadlock state, detect it, and then recover.
  3. Ignore the problem altogether and pretend that deadlocks never occur in the system.





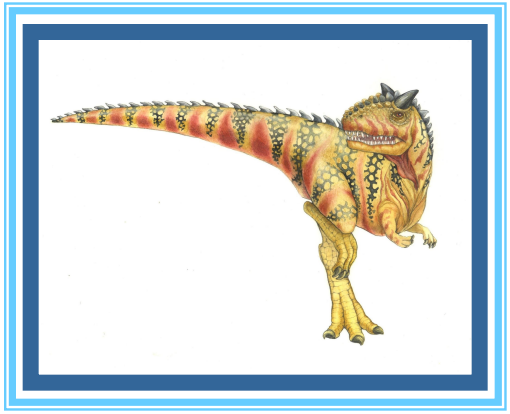
Exercises

From Silberchatz, Operating System Concepts
Chapter 7 Exercises (page: 335 - 338)

- 7.1
- 7.6
- 7.7
- 7.8
- 7.9
- 7.10
- 7.12
- 7.13
- 7.14



End of Chapter 7



Silberschatz, Galvin and Gagne ©2013